LLM-Based Anomaly Detection in System Logs

Lam Christopher Nguyen
la815794@ucf.edu
University of Central Florida
Orlando, Florida, USA

Abstract

In this project, LLMs will be applied in a software application to identify abnormal patterns in network and system logs. The practical application that was deveoped to test LLM anomaly detection is a web-based system designed using the MongoDB, Express, React, and Node (MERN) stack. The framework that will be used to scan the network logs is called LangChain combined with Claude. The topic number in this syllabus is: 14. LLM-Based Anomaly Detection in System Logs. So far the application has been built with Backend Logging using the Javascript logging Library Winston which so far is able to log backend API requests and responses. LangChain framework has been implemented to some extent and is able to scan logs. However more fine-tuning needs to be done as it is not consistent and there are still bugs and issues with the Claude API.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; *Neural networks*; • **Security and privacy** → **Intrusion/anomaly detection and malware mitigation**; • **Software and its engineering** → System administration.

Keywords

large language models, anomaly detection, log analysis, system monitoring, LangChain, natural language processing, security analysis, network logs

ACM Reference Format:
Lam Christopher Nguyen. 2026. LLM-Based Anomaly Detection in System Logs. In . ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Code Availability

The source code for the implementation of this web application can be found at:

https://github.com/lamcnguyen89/CAP_6640_AI_Logging

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2 Introduction and Problem Statement

2.1 Motivation

The motivation for this project is to explore how LLMs can be used to identify problems in system and network logs, which can be a valuable tool for system administrators and security professionals.

2.2 Significance

The significance of this project is that it can help to improve the efficiency and effectiveness of log analysis, which can lead to faster identification of issues and improved security.

2.3 Objectives

The objectives of this project are to develop a web application that can scan network logs using LangChain and Claude, and to evaluate the performance of the application in identifying anomalies in the logs.

3 Related Work

Recent advances in applying large language models to log analysis have shown promising results.

Egersdoerfer et al. [1] began using LLMs such as ChatGPT for log analysis since they are able to retain context and formulate insightful responses over conversations. These LLMs are able to conduct few-shot and in-context learning with reasoning ability. Previously there were several problems with pre-existing techniques. The first issue was that there was a heavy reliance on expert-labeled logs to discern anomalous patterns. This manual labelling slowed down training. The second issues was that they relied on numeric model prediction based on numeric vector input which is not human readable. This lack of human readability rules out targeted error correction by humans.

Qi et al. [4] introduced LogGPT. This paper explored the use of ChatGPT for log-based anomaly detection. It demonstrated that ChatGPT could be effectively used to identify anomalies in system logs, showing promising results in terms of accuracy and efficiency compared to traditional methods.

Guan et al. [2] introduced LogLLM.Traditional deep-learning methods often struggle to capture semantic information in log messages which are typically organized in natural language. LogLLM employed BERT for extracting semantic vectors from log messages, and uses Llama for classifying log sequences. It also uses a projector to align the vector spaces of BERT and Llama to ensure consistency in understanding log semantics. LogLLM doesn't require log parsers to extraxt templates from logs.

He et al. [3] introduced LLMeLog. This paper introduced a system intended to solve two problems. The first problem is that semantic noise is buried in log events which lead to inevitable gaps between the semantics from the logs and their actual intended meaning.

The second problem was that there exists a gap between general semantic embeddings and the specific embedding requirement for anomaly detection tasks. LLMeLog uses LLMs to enrich the content of log events with in-context learning techniques. These enriched logs are then used to fine-tune a pre-trained BERT model. Finally, it trains a transformer-based anomaly detection model with the event representationas produced by this fine-tuned BERT model. F1 scorese exceeded 99 percent and even with using only 10 percent of the labeled data, was still able to achieve an F1 score of 90 percent.

4 Methodology

In order to demonstrate how LLMs can be used to identify problems in system and network logs, a web application will be developed using the MERN stack. The application will then utilize some LLM log frameworks that were developed for LLM log analysis to scan a repository of logs and identify anomalies. An attempt will be made to balance performance with cost efficiency. The performance of the application will be evaluated using standard evaluation metrics for anomaly detection which is to identify errors and odd behaviors like excessive logins and failed login attempts.

4.1 Overview of Web Application

Before going into the logging techniques, a discussion will be made about the type of example application that will be built to generate the logs. This is important because the architecture of the application will determine the approach used to create the LLM log-analysis system.

The application that will be used is a web based service built using ther MERN stack. The MERN stack for web development is a popular choice for building web based applications. MERN stands for MongoDB, Express, React and Node. These are various frameworks that handle different parts of the pipeline. Before going into detail about What each of these frameworks do, you need to understand the overall architecture of a web based application. A web based application consists of a front end User Interface. This front end is where a user will interact directly with the application.

It consists of a front-end user interface that the users will directly interact with through a web browser. In this front end section of the application, the user will essentially be able to Create, Read, Update and Delete data. The front end of the software can be compared to the front of a grocery store where customers can look at products, add them to their carts, ask questions to associates and purchase products.

Beneath the front end is the middle layer of the application. This middle layer is the connection between the front end and the back end (Server). In this middle layer, API requests that perform translate the user's interactions into API requests that are sent to the backend server. It is like a middle translation layer that translate the front end user interactions into commands that the backend code and architecture can understand. Using the gorcery store analogy, the middle layer is like the associates that take customers requests for products, complaints and questions and translate them into requests for managers or the warehouse. These middle level associates understand a little of the both customer facing roles and the warehouse roles and are able to translate between the two.

Beneath the middle layer of the application is the backend portion of the code. The backend portion of the code using the analogy of the grocery store is like the warehouse where products are stored and can be retrieved and sent to the front of the store. In terms of software, this is where user data is stored and can be retrieved. It's where things like website articles, banking information, or whatever is stored inside a database. The backend code is also where the API requests from the middle layer are processed and executed.

So with this 3 part architecture of the web application in mind, we can understand how the different parts of the MERN stack fit into this architecture.

4.2 Overview of the MERN Stack

MERN stands for MongoDB, Express, React and Node. These are various frameworks that handle different parts of the pipeline.

First let us discuss the front end framework which is React. React is a javascript library used for building the User Interface of web applications. It is used in applications like Facebook, Instagram and even Amazon. The main takeway about why React is used to build front end applications is because it is designed to build very scalable web applications. React is built in such a way that 1 web page for a user can be automatically generated, or 1 billion web pages for 1 billion users can be automatically generated without having hand code each individual web page. The react code is used to produce the front end user interface that users will interact with.

The next technology out of the MERN stack to discuss is Express. Express is a web application framework that acts as the middle layer of the application. It allows the front end react code to communicate with the back end server code that stores and handles the database as well as other functions that the end user won't necessarily see. Express is used to create something called API endpoints that can speak to the database. API endpoints are like prebuilt messages that allow a specific set of actions to interact with the database. They translate the user's interactions into code that manipulates the database.

The next technology to discuss is MongoDB. MongoDB is a NoSQL database that is used to store data for the application. It is a document-oriented database that stores data in JSON-like format. It is used to store user data, website articles, and other information that the application needs to function. MongoDB can store music, videos, images and other types of data. It is used to store the data that the application needs to function.

The final technology in the MERN stack is Node. Node is a javascript runtime that allows developers to write front end code and back end code in the same Javascript programming language. Without it, developers would have to write front end code in one programming language, and the back end code in another programming language. This streamlines the process of development

In summary, the MERN stack is a set of technologies that are used to build functional web applications that deal with creating, reading, updating and deleting data. But what we now have to consider is the process of logging and log analysis when using these web applications.

4.3 Web Application Logging Setup

The next portion of the project to consider is the logging portion of the project. In software development, logging is the process of recording information about the application's behavior and performance. There are many reasons to implement logging in an application. The first reason is for debugging purposes. Developers need to be able to trace errors and issues.

The second reason is for system monitoring purposes. System administrators and security professionals need to be able to monitor the system for unusual activity or security threats.

The third reason is for performance analysis. Developerrs need to be able to identify bottlenecks in order to optimize the performance of an application. Performance bottlenecks become an issue when an application serves millions of people or when it is run on limited hardware.

The fourth reason is for auditing and compliance purposes. Sensitive software such as medical software, financial software and government software need to be able to keep track of user activity for auditing and compliance purposes.

A discussion of the logging framework from the log generation in the application to the log analysis using LLMs will now be made.

The first step is to actually implement the logging system in the web application. To do this, the Winston Logging Library was used. In the web development world, Winston is one of the most popular due to it's ease of use and versatility. This library was implemented in the backend of the web application to log Express API requests and responses. Winston is able to log API requests and responses in JSON format which is the required format for the LLM framework to parse logs. Winston is also able to save logs to the MongoDB database as well as locally. Also, since it is the most popular web logging framework, it is actively maintained and has a large community which makes it easier to maintain and troubleshoot.

Logs have to be stored in a certain way so that the LLM log analysis framework can parse these logs. First, logs have to be stored in JSON format. Second logs need to have a correlationID linked to a specific user or user session. This will allow the model to trace the user's journey through the application. Finally, relevant logging info such as a timestamp, message, and standard log levels such as info, warning and error need to be included in the logs.

Once the Winston Logging Framework is implemented as a helper function, it has to be called by the Express API endpoints. API endpoints are like predetermined functions that are called when the user interacts with the front end code in order to deal with data stored in the database.

Once this basic logging setup is in place, the application can start generating logs for various events and actions. These logs can then be analyzed using the LLM framework to gain insights into user behavior, system performance, and potential issues.

4.4 LLM Log Parsing and Preparation

The next step is to prepare the logs for analysis by the LLM framework. It is not enough to just send raw logs to the LLM framework. The logs need to be preprocessed and formatted in a way that the LLM can understand. This involves cleaning the logs, extracting relevant information, and structuring the data in a way that is suitable for analysis.

In order to do this, log preparation is broken down into 4 steps. The first step is retrieving logs and processing them. This is an automated service that periodically fetches the generated logs from the database. Once logs have been retrieved from the database for processing in the LLM, it is marked as processed so that it won't be retrieved again which can reduce the accuracy of results. Also logs are retrieved from the database in batches between 100 to 1000. This will prevent the LLM from being overwhelmed. The larger the batch of logs, the more expensive it is to process, but the LLM is able to analyze over a wider context, so batch size has to balance these two factors.

The next step was to implement a strategy to chunk the logs into manageable amounts. It is not cost effective or technically feasible to send a large amount of logs to the LLM. Logs are too granular and there are token limits when using LLMs. In this application, logs are chunked according to the correlationID that is added to the logs based on a particular user session.

Once logs are retrieved and chunked, they have to be embedded. Embedding is the process of converting the logs into a format that can be understood by the LLM. This involves converting the logs into vectors that capture the semantic meaning of the logs. This is done using a pre-trained embedding model that is able to capture the semantic meaning of the logs. The LLM model embedding model that will be used is the OpenAI model specifically designed for embedding named "text-embedding-3-small". This model is accessed via the OpenAI API.

Once the logs have been embedded, they are stored in a vector database. The vector database that is used in this application is named Qudrant. This vector database is used to store the embedded logs in a way that allows for efficient retrieval and analysis by the LLM framework. Qudrant was chosen mainly because it was free and allows local storage of embedded logs. The more premium cloud based vector databases can be very expensive and for the purposes of this project, a free vector database is sufficient.

So why is the Qudrant Database necessary? Why are services like MongoDB which are database services not sufficient for storing log embeddings? The reason for this is that the structure of Qudrant and other Vector Storage Databases are optimized for storing, searching, retrieving and analyzing high-dimensional vector data. Qudrant is able to store single embeddings with metadata that can be searched. It can also batch store multiple embeddings for greater efficiency. Unlike traditional databases, Qudrant can do vector similarity search, basic database filtering by fields such as correlationalId, logLevel and timestamp. It can be set up to collect statistics, do error handling and logging within the database itself and of course do basic CRUD operations. Besides vector functions, Qudrant also has the ability to do natural language searches that can be translated to vector searches. This is useful for the LLM framework to be able to search for specific logs based on natural language queries.

So now the logs have been generated, retrieved, chunked, converted to vectors and stored in a database so that the AI model can analyze the logs and identify anomalies. The next step is to

implement the LLM framework to analyze the logs and identify anomalies.

4.5 LLM Log Analysis

Now that the logs are generated in the application, converted to embeddings and stored in a vector database, they are ready to be analyzed by the LLM framework. The LLM model that will be used for log analysis are the ChatGPT-4 model from OpenAI. Once the AI Agent is implemented, analysis and insights can be generated from the logs. The LLM framework will be able to identify anomalies in the logs such as excessive logins, failed login attempts, and other unusual activity. The LLM framework will also be able to provide insights into user behavior and system performance based on the logs.

For the purposes of this web application, the first step in integrating the AI Agent is to first create the AI Agent service. This service is similar to an API service that can be called by the backend code to analyze logs and return insights. The services are: Vector Similarity Search, Anomaly Detection, Error Pattern Analysis and Conversational Log Query.

Now we get to the key workflow for the LLM Log Analysis. Log Analysis uses the RAG (Retrieval-Augmented Generation) Pattern. RAG is the process of optimizing the output of a large language model, so it references an external knowledge base outside its training data, before generating a response. RAG basically extends the capabilities of an LLM by allowing it to adapt and incorporate new information that it wasn't trained on. However a possible downside is that if the external source is not reliable, it can lead to inaccurate or misleading responses from the LLM. So the quality of the external knowledge base is crucial for the success of RAG.

The workflow for intelligent log analysis is as follows:

User Query -> Embed Query -> Search Qudrant -> Retrieve Top K Logs -> Send Logs to LLM -> LLM Analyzes Logs -> LLM Generates Response -> Return Response to User

In order to implement RAG within this application, there will be 4 phases that have to be implemented. The first phase is the Query Interface. The second phase is the LLM Analysis phase. The third phase is automated anomaly detection. The fourth phase is the conversational interface.

The first phase is the Query Interface and this is the foundation for implementing RAG. This part of the implementation covers the ability to ask questions to the logs in natural language. Other features that will be available in this pahse are time-based filtering, log level filtering (error, warning, info) and correlation ID filtering.

The second phase is the LLM Analysis phase. This is where the Language Model interprets the retrieved logs. This is the phase of RAG where we actually choose and implement the model for analysis. In this application, GPT-4 from OpenAI was chosen. THe reason this model was chosen over other models like Claude was because it cost less per token. Claude seems to be more powerful for things like coding and understanding code, but for log analysis, GPT-4 is sufficient and more cost effective.

THe third phase is the anomaly detection by the LLM model. In this phase we implement classes for anomaly detection such as detecting statistical anomalies, detecting error clusters and detecting unusual behavior. This is the phase where the LLM model is able to identify anomalies in the logs such as excessive logins, failed login attempts, and other unusual activity.

Anomaly detection is done using a few basic techniquess. In order to detect statistical anomalises, Vector Distance Analysis is done. Logs far from cluster centers in the vector space are flagged as potential anomalies. In order to to detect error clusters, time-series analysis is done. If there is a sudden spike in error rates over a relatively short period of time, this is flagged as a potential incidence. Finally detecting unusual behavior can be done through Correlation ID tracking. If the flow of requests for a particular user session tracked through the correlation ID is broken, this can be flagged as a system issue.

The final phase of the RAG implementation is the conversational interface for interacting with the LLM as it analyzes the logs. For a web application that faces the public, this chat feature for debugging would be hidden behind an administrator dashboard or something of the sort. This conversational interface allows users to ask questions about the logs in natural language and receive insights and analysis from the LLM model. This is a key feature for making the log analysis accessible and user-friendly for system administrators and security professionals who may not have a technical background in log analysis.

4.6 Evaluation Metrics

The performance of the application will be evaluated using standard evaluation metrics for anomaly detection, such as precision, recall, and F1 score. These metrics will be calculated based on the number of true positives, false positives, and false negatives identified by the application.

For review, precision is the ratio of true positives to the total number of positive predictions made by the model. Recall is the ratio of true positives to the total number of actual positive cases in the dataset. The F1 score is the harmonic mean of precision and recall, providing a single metric that balances both aspects of performance.

However, scoring with precision, recall and F1 scores might not be so straight forward for anomaly detection. What is considered a problem and what is not an issue cannot be easily defined. Log Analysis is not only about spotting a problem, but spotting something before it becomes a problem. So the evaluation of the application will also include a qualitative analysis of the insights generated by the LLM model based on the logs. This will involve manually reviewing the insights generated by the model and assessing their relevance and usefulness in identifying potential issues and improving system performance.

5 Evaluation and Results

So far the application has been built with Backend Logging using the Javascript logging Library Winston which so far is able to log backend API requests and responses. LangChain framework has been implemented to some extent and is able to scan logs. However more fine-tuning needs to be done as it is not consistent and there are still bugs and issues with thee Claude API.

You can see in Figure 1 the user interface of the application. The application is essentially a web application that keeps track of experiments and experiment data. It is a standard web application

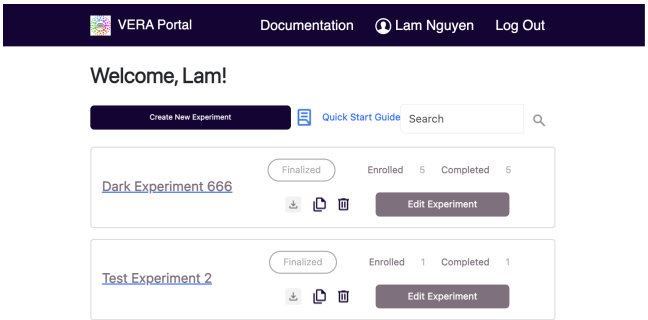

Figure 1: Application UI

with Create, Read, Update and Delete functionality. Data is stored in a database and logs are generated everytime there is communication between the front end of the code which is the website and the backend server.

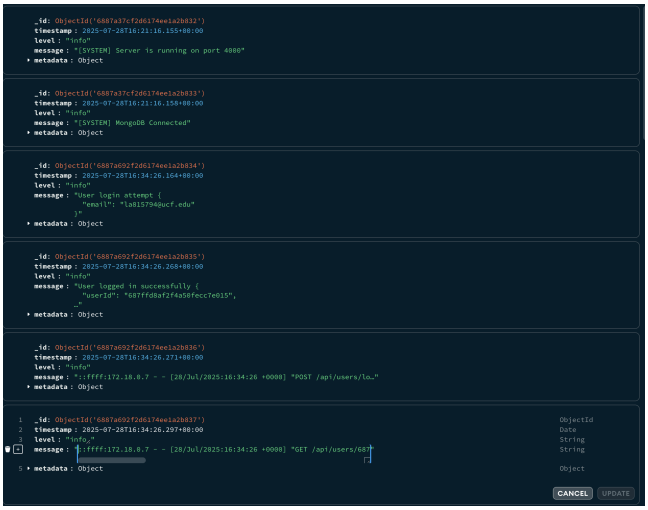

Figure 2: Backend Logs saved in a single location

In Figure 2 you can see that the logs are generated and saved in a single location in a database. The logs are generated using the Winston logging library and are in JSON format. The logs include information about the API requests and responses, as well as any errors that may occur. They are also stored in a separate CSV file that is used for the LangChain framework to scan and analyze the logs. This might not be the most efficient, but for the purposes of this project where we are focused more on implementing LLM Log Analysis, this is sufficient.

In Figure 3 you can see the real-time logs that are generated in the terminal when the application is running. These logs provide information about the API requests and responses, as well as any errors that may occur. They are also used for debugging purposes during development.

The next steps will be to continue fine-tuning the integration of the LangChain framework and the Claude API to improve function,

Figure 3: Real-Time Logs in Terminal

consistency and reliability. Once the application is able to consistently scan logs and identify anomalies, the next step will be to evaluate its performance using the evaluation metrics mentioned above.

6 Discussion and Challenges

In this section a discussion will be made about the limitations, challenges, trade-offs, and possible improvements.

AI Log Analysis is possible and can be very useful for system administrators and security professionals. However, there are several challenges and limitations that need to be considered when implementing an LLM-based log analysis system.

In the building of this application, the first limitation that was encountered was the cost of using LLMs for log analysis. Each API call incurs a cost, and as the rate of logging increases, for example in massive applications such as Banking Apps, or the large social media websites, the cost of using LLMs for log analysis can become prohibitive. Maybe the ongoing cost of using LLMs for log analysis can eventually be offset by a company if they build their own hardware and infrastructure to run their own LLMs. This cost-to-benefit analysis has to be done between using an API for LLM log analysis,building an in-house LLM log analysis system, or just hiring an experienced security professional and building log/security analysis tools that don't require LLMs.

For smaller applications, the ongoing and added cost of accessing an API like ChatGPT for log analysis might not be worth the cost in tokens, the cost in hardware and/or the extra time and resources needed to build the LLM analysis system. Also, the added code complexity of building out this system might in itself introduce more bugs and issues that can lead to more problems than it solves. So for smaller applications, it might be more cost effective to just use traditional log analysis methods and tools without LLMs.

For applications that need to be disconnected from the internet, connecting to an API is not feasible, and an in-house solution would be required. For small and efficient applications run on edge devices, having an entire LLM system versus some light weight log visualization tool might not be worth it.

However, for larger applications that generate a massive amount of logs, and for compnaies that are willing to build inhouse hardware and systems, the benefits can be worth it. Logs are not especially complex to analyze for a human, but the sheer volume of logs can make it impossible for a human to analyze them. LLMs can be used to analyze these logs and identify patterns and anomalies that

might be missed by traditional log analysis methods. LLMs can use relatively simple methods such as vector distance analysis, time-series analysis and correlation ID tracking to identify anomalies in the logs. And if the logs are structured consistently, and correctly with the right information that is standardized and easily parsed, then the LLM is not being asked to do anything too complex; it is more of an issue with the sheer volume of logs.

7 Concluding Remarks

References

[1] Chris Egersdoerfer, Di Zhang, and Dong Dai. 2023. Early Exploration of Using ChatGPT for Log-based Anomaly Detection on Parallel File Systems Logs. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (Orlando, FL, USA) *(HPDC '23)*. Association for Computing Machinery, New York, NY, USA, 315–316. doi:10.1145/3588195.3595943

[2] Wei Guan, Jian Cao, Shiyou Qian, Jianqi Gao, and Chun Ouyang. 2025. LogLLM: Log-based Anomaly Detection Using Large Language Models. arXiv:2411.08561 [cs.SE] https://arxiv.org/abs/2411.08561

[3] Minghua He, Tong Jia, Chiming Duan, Huaqian Cai, Ying Li, and Gang Huang. 2024. LLMeLog: An Approach for Anomaly Detection based on LLM-enriched Log Events. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. 132–143. doi:10.1109/ISSRE62328.2024.00023

[4] Jiaxing Qi, Shaohan Huang, Zhongzhi Luan, Shu Yang, Carol Fung, Hailong Yang, Depei Qian, Jing Shang, Zhiwen Xiao, and Zhihui Wu. 2023. LogGPT: Exploring ChatGPT for Log-Based Anomaly Detection. In *2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*. 273–280. doi:10.1109/HPCC-DSS-SmartCity-DependSys60770.2023.00045